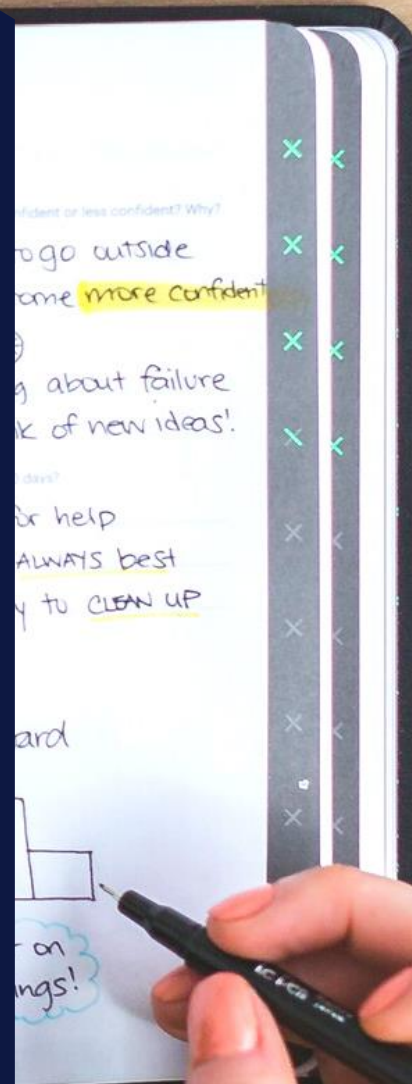




Tarea # 1

Unidad 4. Gestión de Dependencias



Actividades

Actividad: Implementación de Programación Concurrente en la Aplicación de Gestión de Contactos

Descripción de la actividad:

En esta actividad se deberá migrar la aplicación de Gestión de Contactos a un nuevo proyecto basado en Maven o Gradle, integrando lo aprendido en las unidades anteriores y aplicando gestión avanzada de dependencias. Se explorará el uso de repositorios externos para mejorar la interactividad, el manejo de archivos JSON y la correcta administración de versiones y exclusión de dependencias transitivas.

Para desarrollar esta actividad se debe cumplir con los siguientes requerimientos funcionales:

1. Migración del Proyecto a Maven o Gradle:
 - a. Crear un nuevo proyecto en Maven o Gradle e integrar la aplicación de Gestión de Contactos evolucionada en las unidades anteriores.
 - b. Configurar correctamente el archivo pom.xml (Maven) o build.gradle (Gradle).
2. Investigación y Auditoría de Dependencias
 - a. Investigar repositorios externos para mejorar la interactividad y la apariencia de la aplicación.
 - b. Verificar la confiabilidad y seguridad de cada repositorio antes de incluirlo en el proyecto (adjuntar en el informe la auditoría realizada a cada dependencia).
 - c. Implementar bibliotecas que agreguen diseño moderno y funcionalidades interactivas (Ejemplo: FlatLaf para interfaz gráfica, Gson/Jackson para manejo de JSON).
3. Manejo de Serialización y Carga de JSON
 - a. Implementar la funcionalidad para serializar y deserializar contactos en formato JSON.
 - b. Permitir la importación de contactos desde un archivo JSON externo.
4. Gestión de Versiones y Exclusión de Dependencias Transitivas
 - a. Configurar versiones estables en las dependencias

- b. Identificar y excluir dependencias transitivas innecesarias para evitar sobrecarga en el proyecto.
- c. En caso de requerir versiones antiguas de alguna dependencia, configurarlas adecuadamente en el proyecto.

Bibliografía:

- Schildt, H. (2022). **Java: The Complete Reference, Twelfth Edition**. McGraw-Hill.
- Cooper, A., Reimann, R., Cronin, D., & Noessel, C. (2014). **About Face: The Essentials of Interaction Design**. John Wiley & Sons.

Entrega:

Se deberán entregar los siguientes archivos:

- Un informe en PDF con:
 - o Explicación de cada mejora aplicada.
 - o Justificación de los repositorios elegidos y sus beneficios.
 - o Auditoría realizada a cada dependencia
 - o Explicación del manejo de versiones y exclusión de dependencias transitivas.
 - o Capturas de pantalla de la funcionalidad implementada (carga/exportación de JSON)
- Código fuente en un archivo .zip, con comentarios detallando la configuración de dependencias.
- Un Video corto (máximo 3 minutos) donde el estudiante muestre el funcionamiento de la aplicación.

Nombre del fichero: “primerApellido_primerNombre_siglasAsignatura_U#”, ejemplo:
Lopez_Juan_ProgInterfacesG_U4

Rúbrica:

Criterios	Nivel Bajo	Nivel Medio	Nivel Alto	Sub-Puntajes
Migración a Maven o Gradle	No se realizó la migración correctamente. 1	Se realizó, pero con errores de configuración. 2	Migración correcta y configuración bien estructurada. 3	3
Auditoría y selección de repositorios	No se investigaron repositorios o se usaron sin justificación. 0	Se incluyeron repositorios, pero sin auditoría detallada. 1	Se seleccionaron repositorios relevantes con análisis de seguridad y compatibilidad. 2	2
Implementación de manejo de JSON	No se implementó la funcionalidad. 0	Se implementó, pero con errores en la serialización/carga. 1	Funcionalidad de JSON bien implementada y probada. 2	2
Gestión de versiones y exclusión de dependencias	No se realizó un manejo adecuado de versiones. 0	Se aplicó, pero con errores en la compatibilidad. 1	Se aseguraron versiones estables y se excluyeron dependencias innecesarias correctamente. 2	2
Documentación y justificación del diseño	No se presentó documentación o está incompleta. 0	Documentación poco detallada. 1	Informe bien estructurado, con justificación técnica detallada. 2	2
Calidad del código y optimización	Código desorganizado, sin comentarios ni estructura. 0	Código funcional pero con oportunidades de optimización. 1	Código limpio, estructurado y optimizado con buenas prácticas. 2	2
Totales				13pts